CMP9135 Computer Vision

Vishal Maisuria
MSc Artificial Intelligence
University of Lincoln
26439978@students.lincoln.ac.uk

I. Introduction

Segmenting objects cleanly from cluttered scenes is a core problem in computer vision and sits behind many real-world systems, from medical imaging to robotics (Szeliski, 2010). Here, the target is a coloured parachute moving through a dark sky, with 51 RGB frames and a matching set of binary ground-truth masks. The Dice Similarity Coefficient is also used to measure accuracy, which is a standard overlap score for segmentation (dice - Sørensen-Dice similarity coefficient for image segmentation - MATLAB, n.d.).

II. Methodology

A. Pipeline

For each RGB image, the algorithm follows four steps

- Preprocess the image and capture a colour representation
- Build a parachute likelihood image from the colour cues
- Perform region growing from an automatically chosen seed region
- Apply morphological post-processing and connected component analysis to keep a single final object.

This strucutre fits within the traditional region-based segmentation family. Low level cues first mark likely foreground pixels, then morphology and connected components regularise the shape and remove clutter (Parvati Et al., 2008). Automatic operations such as adaptive thresholding and seed selection are used wherever possible so that the method depends only weakly on the specific brightness of each frame (Mangal Et al., 2024).

B. Preprocessing and colour representation

Each input frame is read as an RGB image, converted to double precision in the range [0,1], and lightly smoothed with a Gaussian filter. The smoothing supresses fine-scale noise without removing the parachute edges, which is important before thresholding or region growing (Yamaguchi Et al., 2023).

Rather than converting to CIE LAB, the implementation stays in RGB, but constructs a colourfulness measure that is tailored to this dataset. R, G, and B represent the red, green and blue channels. The parachute is strong in red and green, but weak in blue, whereas the sky remains grey and blueish. To utilise this, a colour score is defined as

$$C = \max(R, G) - B + 0.6\ |R - G|$$

The first term highlights pixels that are brighter in read or green than in blue, and the second term favours locations where the red-green difference is large, which is true on the stripes of the parachute. The combined map is then scaled into [0,1] using min-max normalisation. Using colour distances in this way matches the idea that perceptual differences in colour can help separate objects from the background (Sharma Et al., 2005).

C. Spatial prior and intiial thresholding

Inspection of the sequence shows that the parachute remains roughly in the middle of the frame. The parachute is never on the very left or right, and never reaches the top and bottom borders. This can be encoded as a spatial prior by defining a rectangular region of interest (ROI) covering the central band of the image both horizontally and vertically. Pixels that are ouside this ROI are set to zero in the colour score map so that later stages ignore these areas.

Within this ROI an initial binary mask is produced using MATLAB's adaptive thresholding function followed by binarisation. Adaptive thresholding estimates a threshold from local statistics rather than from the global histogram, making it more robust to the strong illumination gradient caused by the light beam (Helmy, 2023). After thresholding, very small connected components are discarded with an area filter to remove isolated noise blobs and small specular highlights.

D. Seed selection and geodesic region growing

The threholded mask typically contains several surviving blobs. Some of these correspond to parts of the parachute, but others can be small pieces of the cave or light shaft. To pick a good starting point for region growing, connected component analysis is applied and a handful of shape features are computed for each component, such as area, eccentricity, perimeter and circularity $4\pi A/P^2$ (Doube, n.d.). A simple score then favours components that have an area in the expected range for the parachute, are compact, which is high circularity and those that are not extremely elongated, with a low eccentricity. The component with the highest score is used as a seed. If this seed is extremely small, it is dilated to give region growing a more stable starting set.

To obtain an appropriate object region, the method uses geodesic region growing on the colour score image. A geodesic distance map is computed using MATLAB's fast marching-based tools, where pixels similar in colour to the seed have low cost and dissimilar pixels have high cost. Thresholding this map produces a region of pixels that can be reached from the seed via paths with small accumu-

lated colour differences. This is consistent with region-based segmentation schemes where similarity is measured along spatial paths rather than per-pixel only (Parvati Et al., 2008).

E. Morphological refinement and final object selection

The grown region is binarised and refined with standard morphological operations. Small components are removed using area opening, internal gaps are filled with hole filling, and a mild opening operation smooths irregular boundaries (Parvati Et al., 2008). These steps are important to suppress stray pixels that region growing may have picked up while still keeping the overall canopy shape intact.

Finally, connected-component analysis is appied once more. For each remaining component, a compactness-based score similar to the seed selected step is computed, combining area, circularity and eccentricity. Only the highest scoring component is kept as the final parachute mask for that frame, reflecting the prior knowledge that only one parachute is present. If needed, the mask is resized with nearest-neighbour interpolation so that it matches the resolution of the ground-truth mask before evaluation (Taha and Hanbury, 2015).

III. Evaluation and Results

A. Setup

The dataset consists of 51 RGB images and 51 ground-truth masks. Each ground-truth mask is binarised using Otsu's method so that it has a clean {0,1} representation regardless of the encoding used in the files (Helmy, 2023). For each image, the algorithm produces a binary prediction M, which is compared against its ground-truth counterpart S using the dice similarity coefficient

$$DSC(M,S) = \frac{2|M \cap S|}{|M|+|S|}$$

The DSC ranges from 0 to 1, where 1 indicates a perfect overlap, and 0 indicates no overlap. This is beneficial because it summarises precision and recall into a single score (Taha and Hanbury, 2015). I compute a DSC value for each of the 51 images and summarise performance using the mean and standard deviation.

B. Quantitative performance

Using this RGB-based pipeline, the frames achieved DSC values above 0.8, with the best cases exceeding this, meaning that the predicted masks overlap the ground truth almost perfectly apart from minor boundary differences. Across the full sequence and seen in Fig. 1, the DSC values span a range, from 0.00 in the worst case to above 0.8 in the best case. This shows that while the method is generally reliable, it still struggles on frames in the mid-sequence.

C. Best and Worst examples

To better understand the performance, I visualised the five best and five worst frames according to the DSC. For each one, I show the original RGB image, the ground-truth mask and the predicted mask next to each other, as seen in Fig. 2 and Fig. 3.

In Fig. 2, the predicted masks closely follow the parachute, as it moves across the scene and underneath the light. Fig. 5 shows the DSC values for these five frames, with all of them above 0.7, indicating that the program captures both the position and shape of the parachute well with only minor issues.

In Fig. 3, the five worst examples illustrate these failures in more detail. The ground-truth masks occupy the small region that is the parachute, but the predicted masks focus on the large bright area around the parachute, namely the bright light beam. These frames all have a DSC value of below 0.2, highlighting the limitations of using only colour and local region growing, particularly in situations where the foreground is small and surrounded by bright areas.

IV. Appendix

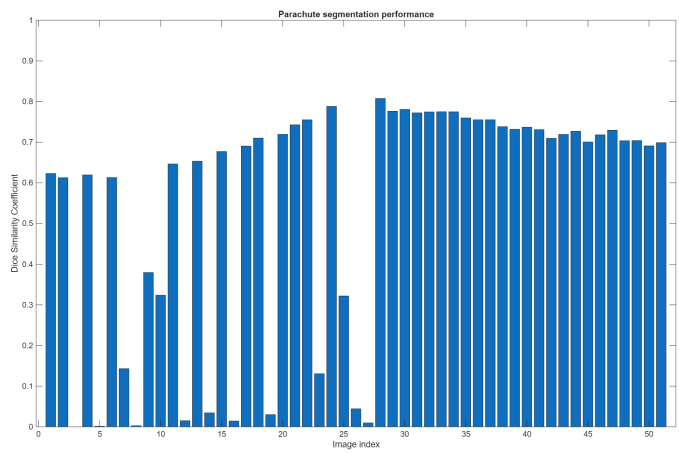

Fig. 1. Parachute Segmentation Performance

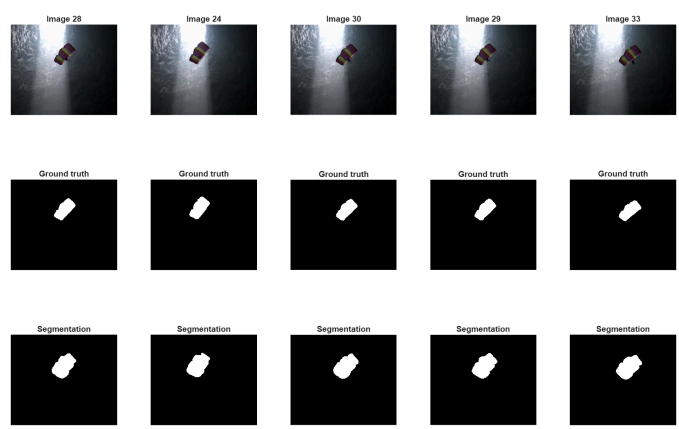

Fig. 2. Best performing frames

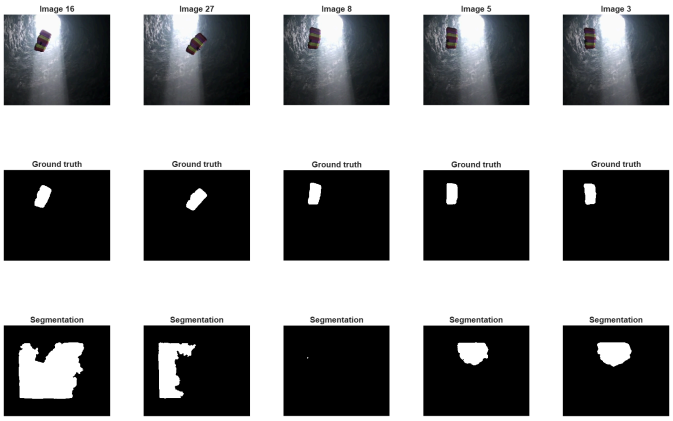

Fig. 3. Worst performing frames

Image 1: DSC = 0.6231
Image 2: DSC = 0.6127
Image 3: DSC = 0.0000
Image 4: DSC = 0.6196
Image 5: DSC = 0.0015
Image 6: DSC = 0.6131
Image 7: DSC = 0.1431
Image 8: DSC = 0.0028
Image 9: DSC = 0.3795
Image 10: DSC = 0.3237
Image 11: DSC = 0.6466
Image 12: DSC = 0.0152
Image 13: DSC = 0.6534
Image 14: DSC = 0.0344
Image 15: DSC = 0.6772
Image 16: DSC = 0.0144
Image 17: DSC = 0.6906
Image 18: DSC = 0.7102
Image 19: DSC = 0.0301
Image 20: DSC = 0.7194
Image 21: DSC = 0.7429
Image 22: DSC = 0.7551
Image 23: DSC = 0.1305
Image 24: DSC = 0.7881
Image 25: DSC = 0.3219
Image 26: DSC = 0.0443
Image 27: DSC = 0.0097
Image 28: DSC = 0.8077
Image 29: DSC = 0.7764
Image 30: DSC = 0.7807
Image 31: DSC = 0.7723
Image 32: DSC = 0.7745
Image 33: DSC = 0.7750
Image 34: DSC = 0.7748
Image 35: DSC = 0.7596
Image 36: DSC = 0.7552
Image 37: DSC = 0.7552
Image 38: DSC = 0.7383
Image 39: DSC = 0.7319
Image 40: DSC = 0.7371
Image 41: DSC = 0.7310
Image 42: DSC = 0.7097
Image 43: DSC = 0.7193
Image 44: DSC = 0.7270
Image 45: DSC = 0.7007
Image 46: DSC = 0.7182
Image 47: DSC = 0.7296
Image 48: DSC = 0.7036
Image 49: DSC = 0.7041
Image 50: DSC = 0.6910
Image 51: DSC = 0.6989

Mean DSC: 0.5505
Std DSC: 0.2896

Fig. 4. DSC values for each frame, with mean and standard deviation

```
Best 5 images (index : DSC)
        28.0000    0.8077
        24.0000    0.7881
        30.0000    0.7807
        29.0000    0.7764
        33.0000    0.7750


Worst 5 images (index : DSC)
        16.0000    0.0144
        27.0000    0.0097
         8.0000    0.0028
         5.0000    0.0015
         3.0000         0
```

Fig. 5. DSC values contd.

```
function CMP9135_26439978()

  %User paths / settings
  rootDir     = 'parachute';        % main folder
  imgDir      = fullfile(rootDir,'images');
  gtDir       = fullfile(rootDir,'GT');
  imgExt      = '*.png';
  gtExt       = '*.png';            % ground-truth extension
  outMaskDir  = fullfile(rootDir,'results_masks');
  if ~exist(outMaskDir,'dir'); mkdir(outMaskDir); end

  imgFiles = dir(fullfile(imgDir,imgExt));
  gtFiles  = dir(fullfile(gtDir,gtExt));

  if numel(imgFiles) ~= numel(gtFiles)
     error('Number of images and ground-truth masks does
not match.');
  end

  nImages  = numel(imgFiles);
  dscVals  = zeros(nImages,1);

  %Main processing loop
  for k = 1:nImages
     % read RGB image
     I =
im2double(imread(fullfile(imgDir,imgFiles(k).name)));

     % read corresponding ground truth and binarise
     GT = imread(fullfile(gtDir,gtFiles(k).name));
     if size(GT,3) > 1
        GT = rgb2gray(GT);
     end
     GT = imbinarize(GT);

     %automatic parachute segmentation
     M = segment_parachute(I);

     % save mask (optional, for appendix)
     [~,baseName,~] = fileparts(imgFiles(k).name);
     imwrite(M, fullfile(outMaskDir, [baseName
'_mask.png']));

     %Dice Similarity Coefficient
     dscVals(k) = dice_coefficient(M,GT);

     fprintf('Image %2d: DSC = %.4f\n',k,dscVals(k));
  end
```

Listing 1. MATLAB code

```
    %statistics & plots
    meanDSC = mean(dscVals);
    stdDSC  = std(dscVals);
    fprintf('\nMean DSC: %.4f\n',meanDSC);
    fprintf('Std  DSC: %.4f\n',stdDSC);

    % bar chart of DSC values
    figure;
    bar(1:nImages, dscVals);
    xlabel('Image index');
    ylabel('Dice Similarity Coefficient');
    title('Parachute segmentation performance');
    ylim([0 1]);

    %find 5 best and 5 worst for report appendices
    [sortedVals, idx] = sort(dscVals,'descend');
    bestIdx  = idx(1:5);
    worstIdx = idx(end-4:end);

    fprintf('\nBest 5 images (index : DSC)\n');
    disp([bestIdx, dscVals(bestIdx)]);

    fprintf('\nWorst 5 images (index : DSC)\n');
    disp([worstIdx, dscVals(worstIdx)]);

    %display best and worst examples
    show_examples(imgDir, gtDir, imgFiles, gtFiles, bestIdx,
'Best results');
    show_examples(imgDir, gtDir, imgFiles, gtFiles, worstIdx,
'Worst results');
end

%Parachute segmentation
function M = segment_parachute(I)
    % I: RGB in [0,1]
    % M: logical mask of parachute

    [h,w,~] = size(I);

    %1. Colour-based likelihood
    I = imgaussfilt(I,1);
    R = I(:,:,1); G = I(:,:,2); B = I(:,:,3);

    chroma1 = max(R,G) - B;      % colourful vs grey
    chroma2 = abs(R-G);          % red/green difference
    colourScore = chroma1 + 0.6*chroma2;
    colourScore = mat2gray(colourScore);

    %2. Spatial prior (where parachute can be)
    % Vertical range: middle 60% (avoid top & bottom
artefacts)
    topV    = round(0.20*h);
    botV    = round(0.80*h);

    % Horizontal range: middle 70%
    leftH   = round(0.15*w);
    rightH  = round(0.85*w);

    spatialMask = false(h,w);
    spatialMask(topV:botV, leftH:rightH) = true;
    colourScore(~spatialMask) = 0;

    %3. Initial adaptive threshold
    T  = adaptthresh(colourScore,0.40);
    bw = imbinarize(colourScore,T);

    bw = bwareaopen(bw,40);  % remove very small specks

    %4. Choose best seed component
    cc = bwconncomp(bw);
    if cc.NumObjects == 0
        M = false(h,w);  return;
    end

    stats =
regionprops(cc,'Area','Eccentricity','Perimeter','Centroid');
    scores = zeros(cc.NumObjects,1);
    for i = 1:cc.NumObjects
        a   = stats(i).Area;
        ecc = stats(i).Eccentricity;
        p   = stats(i).Perimeter + eps;
        circ = 4*pi*a / (p^2);

        % area prior: parachute is small fraction of image
        areaPrior = exp(-((a/(0.006*numel(bw)) - 1).^2));  % peak
near ~0.6% of image

        % compactness prior
        scores(i) = areaPrior * circ * (1 - ecc);
    end
    [~,bestIdx] = max(scores);

    seedMask = false(size(bw));
    seedMask(cc.PixelIdxList{bestIdx}) = true;

    % if seed too tiny, slightly dilate before refinement
    if nnz(seedMask) < 20
        seedMask = imdilate(seedMask, strel('disk',2));
    end
```

```
    %5. Region growing on colourScore
    % Use graydiffweight + imsegfmm to expand around best seed
    distMap = graydiffweight(colourScore, seedMask, 'RolloffFactor',0.7);
    % threshold on geodesic distance
    RG = imsegfmm(distMap, seedMask, 0.045);  % 0.03-0.06 is a reasonable window
    M  = RG > 0;

    %6. Final shape cleanup
    M = bwareaopen(M, 80);
    M = imfill(M,'holes');
    M = imopen(M, strel('disk',2));

    % keep most compact component after growth
    cc2 = bwconncomp(M);
    if cc2.NumObjects == 0
        M = false(size(M));  return;
    end
    stats2 = regionprops(cc2,'Area','Perimeter','Eccentricity');
    score2 = zeros(cc2.NumObjects,1);
    for i = 1:cc2.NumObjects
        a   = stats2(i).Area;
        p   = stats2(i).Perimeter + eps;
        circ = 4*pi*a/(p^2);
        ecc  = stats2(i).Eccentricity;
        score2(i) = circ*(1-ecc)*log(1+a);
    end
    [~,idx2] = max(score2);
    M = false(size(M));
    M(cc2.PixelIdxList{idx2}) = true;
end

%Dice coefficient

function d = dice_coefficient(M,GT)
    M  = logical(M);
    GT = logical(GT);

    intersection = nnz(M & GT);
    d = 2 * intersection / (nnz(M) + nnz(GT) + eps);
end

%Helper to show example results

function show_examples(imgDir, gtDir, imgFiles, gtFiles, indices, figTitle)

    figure('Name',figTitle);
    n = numel(indices);
    for i = 1:n
        k = indices(i);
        I  = im2double(imread(fullfile(imgDir,imgFiles(k).name)));
        GT = imread(fullfile(gtDir,gtFiles(k).name));
        if size(GT,3) > 1
            GT = rgb2gray(GT);
        end
        GT = imbinarize(GT);

        M  = segment_parachute(I);

        subplot(3,n,i);      imshow(I);  title(sprintf('Image %d',k));
        subplot(3,n,i+n);    imshow(GT); title('Ground truth');
        subplot(3,n,i+2*n);  imshow(M);  title('Segmentation');
    end
end
```

Listing 5. MATLAB code contd.

References

dice - Sørensen-Dice similarity coefficient for image segmentation - MATLAB (n.d.) Available from https://uk.mathworks.com/help/images/ref/dice.html [accessed 20 March 2026].

Dice, L.R. (1945) Measures of the Amount of Ecologic Association Between Species. *Ecology,* 26(3) 297–302. Available from https://onlinelibrary.wiley.com/doi/abs/10.2307/1932409 [accessed 20 March 2026].

Doube, M. (n.d.) Multithreaded two-pass connected components labelling and particle analysis in ImageJ. *Royal Society Open Science,* 8(3) 201784. Available from https://pmc.ncbi.nlm.nih.gov/articles/PMC8074980/ [accessed 20 March 2026].

Helmy, B.E.-D. (2023) *Understanding Otsu's Method for Image Segmentation | Baeldung on Computer Science* Available from https://www.baeldung.com/cs/otsu-segmentation [accessed 19 March 2026].

Image segmentation using Morphological operations in Python (n.d.) Available from https://www.geeksforgeeks.org/python/image-segmentation-using-morphological-operation/ [accessed 20 March 2026].

Mangal, A., Garg, H. and Bhatnagar, C. (2024) A Robust Co-saliency Object Detection Model by Applying CLAHE and Otsu Segmentation Method. *International Journal of Intelligent Systems and Applications in Engineering,* 12(1s) 481–490. Available from https://ijisae.org/index.php/IJISAE/article/view/3485 [accessed 20 March 2026].

Marçal, A. (2023) Normalised Color Distances:. In: *Proceedings of the 3rd International Conference on Image Processing and Vision Engineering.* 2023 Prague, Czech Republic: SCITEPRESS - Science, Technology Publications, 134–141.

Parvati, K., Prakasa Rao, B.S. and Mariya Das, M. (2008) Image Segmentation Using Gray-Scale Morphology and Marker-ControlledWatershed Transformation. *Discrete Dynamics in Nature and Society,* 2008(1) 384346. Available from https://onlinelibrary.wiley.com/doi/abs/10.1155/2008/384346 [accessed 20 March 2026].

Rabinovich, A., Belongie, S., Lange, T. and Buhmann, J. (2006) Model Order Selection and Cue Combination for Image Segmentation. In: *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06).* June 2006 1130–1137.

Sharma, G., Wu, W. and Dalal, E.N. (2005) The CIEDE2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations. *Color Research & Application,* 30(1) 21–30. Available

from https://onlinelibrary.wiley.com/doi/abs/10.1002/col.20070 [accessed 20 March 2026].
Szeliski, R. (2010) *Computer Vision: Algorithms and Applications.*
Taha, A.A. and Hanbury, A. (2015) Metrics for evaluating 3D medical image segmentation: analysis, selection, and tool. *BMC Medical Imaging,* 15 29. Available from https://pmc.ncbi.nlm.nih.gov/articles/PMC4533825/ [accessed 20 March 2026].
Xu, Y., Quan, R., Xu, W., Huang, Y., Chen, X. and Liu, F. (2024) Advances in Medical Image Segmentation: A Comprehensive Review of Traditional, Deep Learning and Hybrid Approaches. *Bioengineering,* 11(10) 1034. Available from https://pmc.ncbi.nlm.nih.gov/articles/PMC11505408/ [accessed 20 March 2026].
Yamaguchi, Y., Yoshida, I., Kondo, Y., Numada, M., Koshimizu, H., Oshiro, K. and Saito, R. (2023) Edge-preserving smoothing filter using fast M-estimation method with an automatic determination algorithm for basic width. *Scientific Reports,* 13(1) 5477. Available from https://www.nature.com/articles/s41598-023-32013-9 [accessed 20 March 2026].